PermitLink: Integrating the Permit Application, Treasury, and Notification Systems of an LGU Business Permit Process

Team Nexora · ITPC 313 – System Integration and Architecture · Prefinal Week 1 Draft

---

Part 1 – Title Page

Project Title: PermitLink: Event-Driven Integration of the Permit Application, Treasury, and Notification Systems for LGU Business Permits

Approved Scenario: Rank 3 – LGU Business Permits

Team Name: Nexora

Course: ITPC 313 – System Integration and Architecture

Instructor: Neal Errol K. Bulawan

Date: October 2026

Team Members and Roles *(proposed assignment: swap names to match what your team agreed on)*

Member	Role
Jose Arym Diaz III	Project Lead / Architect
Princess Haneen Dibarosan	Integration Developer
Jericho M. Fegason	Data and Security Lead
Hannah L. Alcantara	QA and Documentation Lead

Part 2 – Executive Summary

The problem. In a typical Local Government Unit (LGU), a business owner who applies for a permit must first file at the permit office, then walk to the treasury to pay the assessed fees, and then walk back to the permit office to show the official receipt as proof of payment. The two offices run separate systems that do not talk to each other, so staff re-encode payment details by hand. This causes repeated trips, queues, encoding errors, and delays in releasing permits.

The systems. Three systems are involved: the **Permit Application System** (owned by the Business Permits and Licensing Office), the **Treasury System** (owned by the City/Municipal Treasurer's Office), and the **Notification Service** (owned by the LGU's IT/MIS office).

The proposed integration. PermitLink connects the three systems through three flows. (1) *REST:* the Permit Application System calls `GET /payments/{applicationId}` on the Treasury System to check payment status on demand. (2) *Message broker:* the Treasury System publishes a `payment.posted` message the moment a payment is recorded, and the Permit Application System consumes it and updates the application automatically. (3) *Event:* when a permit is approved, the Permit Application System publishes `permit.approved`, and the Notification Service consumes it and sends the applicant a notice. Only synthetic business name, owner name, and payment amount cross system boundaries.

Expected benefit. Applicants no longer carry proof of payment between offices, staff no longer re-encode receipts, and applicants are told automatically when the permit is ready. The team will demonstrate this with measurable targets: payment status reflected in the permit system within 5 seconds of posting, no lost payment messages, and an approval notice sent within 10 seconds of approval.

Part 3 – Background and Problem Statement

3.1 The Organization

The project is modeled on a generic Philippine city or municipal LGU (referred to here as "the LGU"). Business permits are processed by the **Business Permits and Licensing Office (BPLO)**. Fees and local taxes are collected by the **Treasurer's Office**. The Mayor's Office, through its IT/MIS unit, maintains shared digital services such as applicant communication. No real LGU data is used; all records in this project are synthetic.

3.2 The Current Process

1. The applicant submits a business permit application at the BPLO.
2. BPLO staff assess the fees due and give the applicant an order of payment.
3. The applicant goes to the Treasurer's Office, pays, and receives an official receipt.
4. The applicant returns to the BPLO and presents the receipt as proof of payment.
5. BPLO staff manually check the receipt and re-encode the payment details (receipt number, amount, date) into the permit system.
6. The application is approved and the permit is released. The applicant usually learns about it by returning to the office or calling.

3.3 Where Data Is Siloed or Re-encoded by Hand

- **Payment data is siloed.** Payment is recorded only in the Treasury System. The Permit Application System has no way to see it.
- **Proof of payment is re-encoded.** BPLO staff retype receipt details from a paper receipt, which creates risk of typing errors and mismatched amounts.
- **Status is not shared.** The applicant is the "messenger" between the two offices, walking back and forth to carry proof.

- **No automatic notification.** Approval is not communicated unless staff or the applicant follow up manually.

3.4 Problem Statement

Applicants go back and forth between the permit office and the treasury to show proof of payment, because the Permit Application System and the Treasury System are not integrated. This wastes applicants' time, adds manual re-encoding work and error risk for staff, and delays permit approval and release.

Part 4 – Objectives

4.1 General Objective

To design and demonstrate an integration of the Permit Application System, Treasury System, and Notification Service that removes the need for applicants to carry proof of payment between offices, using REST, a message broker, and event-based communication.

4.2 Specific Objectives

1. **Payment verification (REST).** Implement `GET /payments/{applicationId}` on the Treasury System so the Permit Application System can retrieve payment status, with 95% of requests answered within 500 ms at 50 concurrent users.
2. **Automatic payment sync (broker).** Publish a `payment.posted` message from the Treasury System and consume it in the Permit Application System so that 99% of payments are reflected on the application within 5 seconds, with zero lost or duplicated updates in testing.
3. **Approval notice (event).** Publish `permit.approved` from the Permit Application System and have the Notification Service send a notice for 100% of approved test applications within 10 seconds.
4. **Data minimization.** Limit data crossing system boundaries to synthetic business name, owner name, and payment amount (plus the identifiers needed to link records), verified by a payload review with 0 extra personal fields.
5. **Reproducible environment.** Run all three systems and the broker with a single `docker compose up` command, documented in the repository README, so the whole demo starts in under 5 minutes on a standard laptop.

Part 5 – Scope and Limitations

5.1 In Scope

- Three integration flows: REST status check, broker-based `payment.posted`, and event-based `permit.approved` with applicant notice.

- A simplified permit workflow: application submitted → payment pending → payment confirmed → approved → notice sent.
- Integration design deliverables: API contract, message and event schemas, architecture diagrams, and quality attribute test results.
- Error handling for the integration: retries, duplicate messages, and an unreachable Treasury System.
- Role-based access between services (service-to-service authentication) and audit logging of integration events.

5.2 Out of Scope

- Real LGU systems, real applicants, or real payments.
- Full business permit requirements (zoning, sanitary, fire safety, barangay clearance, and other office clearances).
- Real payment gateways, online banking, and e-wallets.
- Assessment and computation of actual local tax and fee schedules.
- Permit renewal, amendment, and closure processes.
- Production deployment, hosting, and long-term maintenance.

5.3 What Will Be Mocked

- **Permit Application System and Treasury System:** simplified prototype services with their own databases and seeded synthetic data.
- **Notification Service channel:** a mock email/SMS sink (for example, a local mail-capture tool or a log) instead of a real telecom or email provider.
- **Identity and payment records:** generated fake business names, owner names, and amounts.

5.4 Limitations

- Results come from a lab environment, so performance numbers will not match a real LGU's infrastructure or network.
- The prototype covers one payment per application and one permit type.
- Security is demonstrated at prototype level and is not a substitute for a formal security audit.

Part 6 – Systems to Be Integrated

System	Owner	Purpose	Technology Stack (proposed)	Data It Holds	Interface It Exposes
Permit Application System	Business Permits and Licensing Office (BPLO)	Receives business permit applications,	Node.js (Express) or Python (FastAPI),	Application ID, synthetic business name, owner	REST API for applications; consumes `payment.post`

System	Owner	Purpose	Technology Stack (proposed)	Data It Holds	Interface It Exposes
		tracks status, checks proof of payment, approves permits	PostgreSQL, Docker	name, assessed amount, application status, payment reference	`ed` from the broker; publishes `permit.approved`
Treasury System	City/Municipal Treasurer's Office	Records payments of permit fees and issues official receipts	Python (FastAPI) or Java (Spring Boot), PostgreSQL, Docker	Application ID, payment amount, payment date, official receipt number, payment status	`GET /payments/{applicationId}` (REST); publishes `payment.posted` to the broker
Notification Service	LGU IT/MIS Office (Mayor's Office)	Sends notices to applicants when events occur	Node.js worker, mock email/SMS sink, Docker	Notification log (event ID, recipient, message, sent time, delivery status)	Consumes `permit.approved` events; no public API needed
Message Broker *(integration infrastructure)*	Shared / LGU IT/MIS Office	Carries `payment.posted` messages and `permit.approved` events reliably between systems	RabbitMQ (queues for broker flow, topic exchange for events), Docker	Messages in transit only	AMQP publish/subscribe

Integration Summary

#	Integration	Style	From → To	Trigger	Payload
1	Payment status check	REST	Permit System → Treasury System	Staff or system needs to	`GET /payments/{applicationId}` returns status,

#	Integration	Style	From → To	Trigger	Payload
				confirm payment	amount, receipt number
2	`payment.posted`	Message broker (durable queue, acknowledged delivery)	Treasury System → Permit System	A payment is recorded	applicationId, amount, receiptNo, postedAt
3	`permit.approved`	Event (publish/subscribe)	Permit System → Notification Service	A permit is approved	applicationId, businessName, ownerName

Data Crossing Boundaries

Only **synthetic business name, owner name, and payment amount** cross system boundaries. Records are linked using the `applicationId` and the official receipt number, which are generated test identifiers and not personal data.

Part 7 – Requirements

7.1 Functional Requirements (Integration Flows and User Stories)

ID	Type	Requirement
FR-1	Integration flow (REST)	The Permit Application System shall call `GET /payments/{applicationId}` on the Treasury System and receive the payment status (`UNPAID`, `PAID`, or `NOT_FOUND`), amount, and receipt number.
FR-2	Integration flow (Broker)	When a payment is recorded, the Treasury System shall publish `payment.posted`, and the Permit Application System shall consume it and mark the matching application as `PAID`.
FR-3	Integration flow (Event)	When a permit is approved, the Permit Application System shall publish `permit.approved`, and the Notification Service shall consume it and send the applicant a notice.
FR-4	User story	As a **business owner**, I want my payment to be recognized automatically by the permit office, so

ID	Type	Requirement
		that I do not have to bring my receipt back and forth between offices.
FR-5	User story	As a **BPLO staff member**, I want to see the payment status on an application without retyping receipt details, so that I save time and avoid encoding errors.
FR-6	User story	As a **treasury clerk**, I want my posted payment to reach the permit system automatically, so that I do not have to issue separate proof for the applicant.
FR-7	User story	As a **business owner**, I want to receive a notice when my permit is approved, so that I know when to claim it without calling or visiting.
FR-8	Integration flow (Error handling)	If a message is received twice, the Permit Application System shall process it once only (idempotent handling).
FR-9	Integration flow (Error handling)	If the Treasury System is unreachable during a REST call, the Permit Application System shall return a clear "payment status unavailable" result and shall not approve the application.
FR-10	Audit	Each integration message and event shall be logged with ID, type, timestamp, and outcome.

7.2 Quality Attribute Scenarios

Each scenario states the source, stimulus, artifact, environment, response, and a measurable response measure.

ID	Attribute	Source	Stimulus	Artifact	Environment	Response	Response Measure
QAS-1	Performance (REST)	Permit Application System	Sends `GET /payments/{applicationId}`	Treasury System REST API	Normal operation, 50 concurrent users	Returns payment status	95% of requests complete within 500 ms
QAS-2	Performance (sync latency)	Treasury clerk	Posts a payment	Broker flow to	Normal operation, 50 payments	Application is	99% of payments reflected

ID	Attribute	Source	Stimulus	Artifact	Environment	Response	Response Measure
				Permit System	per minute	updated to `PAID`	within 5 seconds
QAS-3	Reliability (no message loss)	Treasury System	Publishes `payment.posted` while the Permit System is down for 2 minutes	Broker queue	Partial failure	Message is held and delivered once the Permit System restarts	0 of 100 test messages lost
QAS-4	Reliability (idempotency)	Broker	Redelivers the same `payment.posted` message up to 3 times	Permit Application System	Fault condition	Application is updated once only	0 duplicate payment records in 100 test redeliveries
QAS-5	Performance (notification)	Permit staff	Approves a permit	Notification Service	Normal operation	Applicant notice is sent	95% of notices sent within 10 seconds of approval
QAS-6	Availability (fault isolation)	Operations	Notification Service is stopped	Permit Application System	Partial failure	Permit approval still completes; notice is sent after the service returns	100% of approvals succeed during the outage; queued notices delivered within 30 seconds of recovery
QAS-7	Security (access control)	Unauthorized caller	Calls the Treasury payments API without a valid	Treasury System REST API	Normal operation	Request is rejected and logged	100% of 20 unauthenticated test requests receive HTTP 401

ID	Attribute	Source	Stimulus	Artifact	Environment	Response	Response Measure
			service token				and are logged
QAS-8	Security / Privacy (data minimization)	Reviewer	Inspects all integration payloads	Messages, events, and API responses	Test run	Only approved synthetic fields are present	0 payloads contain fields beyond business name, owner name, payment amount, and link identifiers
QAS-9	Modifiability	Developer	Adds a second notification channel (for example, SMS in addition to email)	Notification Service	Development	Change is made without modifying the Permit or Treasury Systems	0 changes to Permit and Treasury code; completed in no more than 1 working day
QAS-10	Deployability	Team member	Runs `docker compose up` on a clean laptop	Whole prototype	Development	All services start and pass a health check	All services healthy within 5 minutes

7.3 Constraints

Type	Constraint
Budget	No funding; use free and open-source tools only (Docker, RabbitMQ, PostgreSQL, GitHub or GitLab). Target cost: ₱0.
Hardware	Must run on team members' laptops using Docker Compose, without dedicated servers.

Type	Constraint
Time	Must follow the prefinal schedule: proposal parts 1–7 and repository in Week 1, and design work (APIs, events, architecture) from Week 2.
Legal and ethical	Use synthetic or anonymized data only. Real personal data would fall under the Data Privacy Act of 2012 (RA 10173), so none is collected, stored, or transmitted. The design should reflect principles of data minimization and access control.
Process / Policy	The design should support the goal of faster, simpler government transactions in line with the Ease of Doing Business and Efficient Government Service Delivery Act (RA 11032). *(Team to verify the exact provisions before citing them in the final paper.)*
Technical	The integration must use the three specified styles: REST, a message broker, and event-based communication.

Appendix A – Approved Scenario Record (Deliverable 1)

Item	Detail
Rank	3
Scenario	LGU Business Permits
Problem	Applicants go back and forth between the permit office and the treasury to show proof of payment.
Systems	Permit Application System, Treasury System, Notification Service
Integration 1 (REST)	The permit system calls `GET /payments/{applicationId}` on the treasury system.
Integration 2 (Broker)	The treasury publishes `payment.posted`, and the permit system consumes it.
Integration 3 (Event)	The permit system publishes `permit.approved`, and the notification service sends the applicant a notice.
Data crossing boundaries	Synthetic business name, owner name, and payment amount only.
Approval	Approved by the instructor (confirmed in the Wednesday lab session).

Appendix B – Repository README Draft (Deliverable 9)

```
# PermitLink – LGU Business Permits Integration

ITPC 313 – System Integration and Architecture | Team Nexora
Instructor: Neal Errol K. Bulawan

## Overview
Integrates a Permit Application System, a Treasury System, and a
Notification Service so applicants no longer carry proof of payment
between offices.

## Integrations
1. REST:   Permit System -> GET /payments/{applicationId} -> Treasury System
2. Broker: Treasury System publishes payment.posted -> Permit System consumes
3. Event:  Permit System publishes permit.approved -> Notification Service sends
notice

## Data
Synthetic data only (business name, owner name, payment amount).
No real personal data is used.

## Team
- Jose Arym Diaz III – Project Lead / Architect
- Princess Haneen Dibarosan – Integration Developer
- Hannah L. Alcantara – Data and Security Lead
- Jericho M. Fegason – QA and Documentation Lead

## Repository Structure
/docs          proposal, diagrams, consultation minutes
/permit-service
/treasury-service
/notification-service
/docker-compose.yml

## Run
docker compose up

## Status
Prefinal Week 1: proposal parts 1-7 drafted, repository created.
```

Reminder: add the instructor as a member of the repository, then submit the repository link and the Parts 1–7 draft through Google Classroom (one member submits for the team).

Appendix C – Consultation 1 Minutes Template (Thursday, 8:00 AM – 12:00 NN)

Field	Entry
Date / time	

Field	Entry
Attendees	
Items discussed	
Instructor feedback	
Decisions made	
Action items (owner, due date)	

Appendix D – Week 1 Checklist

- ☐ Three candidate scenarios submitted, ranked by preference
- ☐ One scenario approved (LGU Business Permits)
- ☐ Roles assigned (Project Lead / Architect, Integration Developer, Data and Security Lead, QA and Documentation Lead)
- ☐ Proposal parts 1–7 drafted
- ☐ Every quality attribute written as a measurable scenario
- ☐ Repository created and shared with the instructor
- ☐ Consultation 1 minutes kept